# RIGISTRER;

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { UserPlus, Mail, Lock, Loader2 } from "lucide-react";
import { InputOTP, InputOTPGroup, InputOTPSlot } from "@/components/ui/input-otp";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { toast } from "@/components/ui/use-toast";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Register() {
 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [confirmPassword, setConfirmPassword] = useState("");
 const [error, setError] = useState("");
 const [loading, setLoading] = useState(false);
 const [showOtp, setShowOtp] = useState(false);
 const [otpCode, setOtpCode] = useState("");

 const handleSubmit = async (e) => {
  e.preventDefault();
  setError("");
  if (password !== confirmPassword) {
   setError("Passwords do not match");
   return;
  }
  setLoading(true);
  try {
   await base44.auth.register({ email, password });
   setShowOtp(true);
  } catch (err) {
   setError(err.message || "Registration failed");
  } finally {
   setLoading(false);
  }
 };

 const handleVerify = async () => {
  setError("");
  setLoading(true);
  try {
   const result = await base44.auth.verifyOtp({ email, otpCode });
   if (result?.access_token) {
    base44.auth.setToken(result.access_token);
   }
   window.location.href = safeReturnTo();
  } catch (err) {
```

```
    setError(err.message || "Invalid verification code");
   } finally {
    setLoading(false);
   }
  };

  const handleResend = async () => {
   setError("");
   try {
    await base44.auth.resendOtp(email);
    toast({
     title: "Code sent",
     description: "Check your email for the new code.",
    });
   } catch (err) {
    setError(err.message || "Failed to resend code");
   }
  };

  const handleGoogle = () => {
   base44.auth.loginWithProvider("google", safeReturnTo());
  };

  if (showOtp) {
   return (
    <AuthLayout
     icon={Mail}
     title="Verify your email"
     subtitle={`We sent a code to ${email}`}
    >
     {error && (
      <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
       {error}
      </div>
     )}
     <div className="flex justify-center mb-6">
      <InputOTP
       maxLength={6}
       value={otpCode}
       onChange={setOtpCode}
       autoFocus
       autoComplete="one-time-code"
      >
       <InputOTPGroup>
        <InputOTPSlot index={0} />
        <InputOTPSlot index={1} />
        <InputOTPSlot index={2} />
        <InputOTPSlot index={3} />
        <InputOTPSlot index={4} />
        <InputOTPSlot index={5} />
       </InputOTPGroup>
      </InputOTP>
```

```
    </div>
    <Button
     className="w-full h-12 font-medium"
     onClick={handleVerify}
     disabled={loading || otpCode.length < 6}
    >
     {loading ? (
      <>
       <Loader2 className="w-4 h-4 mr-2 animate-spin" />
       Verifying...
      </>
     ) : (
      "Verify"
     )}
    </Button>
    <p className="text-center text-sm text-muted-foreground mt-4">
     Didn't receive the code?{" "}
     <button onClick={handleResend} className="text-primary font-medium hover:underline">
      Resend
     </button>
    </p>
   </AuthLayout>
  );
 }

 return (
  <AuthLayout
   icon={UserPlus}
   title="Create your account"
   subtitle="Sign up to get started"
   footer={
    <>
     Already have an account?{" "}
     <Link
      to={"/login" + (safeReturnTo() !== "/" ? "?returnTo=" +
encodeURIComponent(safeReturnTo()) : "")}
      className="text-primary font-medium hover:underline"
     >
      Log in
     </Link>
    </>
   }
  >
   <Button
    variant="outline"
    className="w-full h-12 text-sm font-medium mb-6"
    onClick={handleGoogle}
   >
    <GoogleIcon className="w-5 h-5 mr-2" />
    Continue with Google
   </Button>
```

```
    <div className="relative mb-6">
     <div className="absolute inset-0 flex items-center">
      <div className="w-full border-t border-border" />
     </div>
     <div className="relative flex justify-center text-xs uppercase">
      <span className="bg-card px-3 text-muted-foreground">or</span>
     </div>
    </div>

    {error && (
     <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
      {error}
     </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-4">
     <div className="space-y-2">
      <Label htmlFor="email">Email</Label>
      <div className="relative">
       <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground" aria-hidden="true" />
       <Input
        id="email"
        type="email"
        autoComplete="email"
        autoFocus
        placeholder="you@example.com"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        className="pl-10 h-12"
        required
       />
      </div>
     </div>
     <div className="space-y-2">
      <Label htmlFor="password">Password</Label>
      <div className="relative">
       <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground" aria-hidden="true" />
       <Input
        id="password"
        type="password"
        autoComplete="new-password"
        placeholder="••••••••"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        className="pl-10 h-12"
        required
       />
      </div>
     </div>
     <div className="space-y-2">
```

```
      <Label htmlFor="confirm">Confirm Password</Label>
      <div className="relative">
       <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground" aria-hidden="true" />
       <Input
        id="confirm"
        type="password"
        autoComplete="new-password"
        placeholder="••••••••"
        value={confirmPassword}
        onChange={(e) => setConfirmPassword(e.target.value)}
        className="pl-10 h-12"
        required
       />
      </div>
     </div>
     <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
      {loading ? (
       <>
        <Loader2 className="w-4 h-4 mr-2 animate-spin" />
        Creating account...
       </>
      ) : (
       "Create account"
      )}
     </Button>
    </form>
   </AuthLayout>
  );
}
```